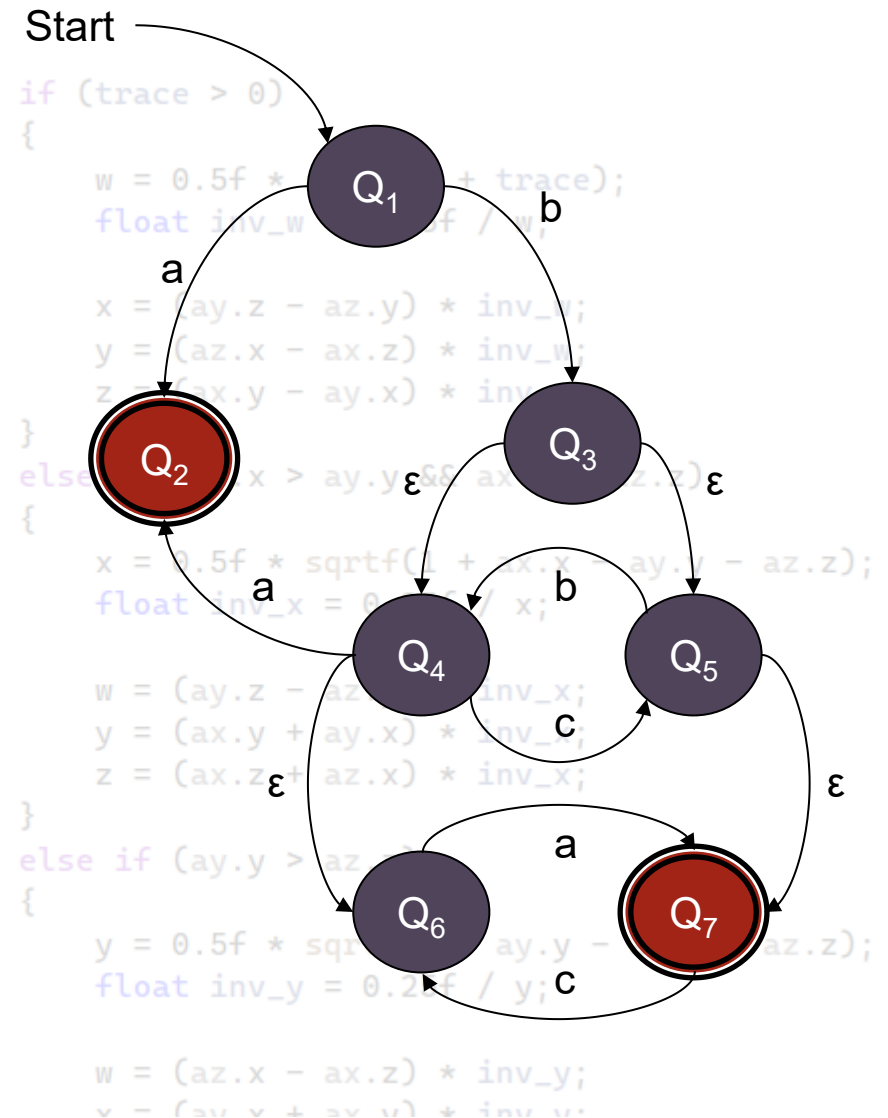


Algorithmen und Datenstrukturen

1 – Einführung

Tobias Werner M. Sc.
Uni Bayreuth - Angewandte Informatik 3
Leibniz-FH Hannover
Duale Hochschule Sachsen
tobias.werner@uni-bayreuth.de



Einleitung

Umfrage

Ziel:

- Erfahren von Vorkenntnissen
- Abgleich mit der Vorlesungsplanung

Vorgehen:

- alle externen Informationsquellen schließen
- Beantworten der Fragen:
 "Was ist aus der C-Programmierung noch bekannt?"
 und
 **"Gegeben eine Liste an Namen und Telefonnummern.
 Wie findet man eine Telefonnummer zu einer Person?"**
- anonyme Umfrage
- mehrere Antworten pro Person möglich

Vortragsübersicht

1 – Einführung

- Organisation
- Motivation
- Inhalt der Vorlesung

- Formalisierung einfacher Daten
- maschinennahe Datenrepräsentation
- maschinenferne Datenrepräsentation

Organisation (1)

Zielgruppe

- Studierende Technische Informatik im 3. Semester

Voraussetzungen

- Theoretische Informatik
- grundlegende Programmierfähigkeiten in C
- Basisverständnis Englisch

Hard- und Software

- halbwegs aktueller PC
- installierter Compiler + IDE
 - Windows: Visual Studio Community
 - Linux, macOS: Qt Creator + GCC

Organisation (2)

Ablauf

pro Kapitel drei Teile:

- eine Vorlesung (ca. 1 Stunde)
- eine Übung (ca. 1 Stunde)
- eine Übungskorrektur (ca. 1 Stunde)

Vorlesung

Folien:

- tagesaktuell im e-Learning-Portal der DHSN
- bei Bedarf Notizen machen, es gibt kein ausformuliertes Skript!

interaktives Lernen:

- Zwischenfragen sind erlaubt und erwünscht
- hin und wieder Mentimeter-Umfragen
- **nach zusätzlicher Pause rufen, wenn Bedarf besteht!**

Organisation (3)

Übung

Aufgaben:

- tagesaktuell im e-Learning-Portal
- in Umfang und Schwierigkeit ähnlich der Klausur

Bearbeitung:

- in festen Gruppen zwischen 2 und 4 Teilnehmern
- Lösung in der Gruppe erstellen
- **bei Bedarf den Dozenten um Hilfe bitten!**

Korrektur:

- auf Wunsch Vorrechnen der eigenen Lösung im Plenum
 - Korrektur durch das Plenum und den Dozenten
 - sonst Vorrechnen durch den Dozenten
- evtl. Musterlösungen für alle Übungsaufgaben kurz vor der Klausur

zudem: Probeklausur mit Musterlösung kurz vor Kursende

Organisation (4)

Klausur

Termin: dem Dozenten nicht bekannt

Ablauf: schriftlich, 2 h Schreibdauer

Hilfsmittel: vermutlich keine

nach Abstimmung im Kurs möglich:

→ ein DIN-A4-Zettel handschriftlich mit Notizen

→ dann aber keinerlei Auswendiglernfragen!

Benotung:

- ca. 50 Punkte aus ca. 5 Aufgaben
- Zweit- und Drittversuche werden nicht leichter

Inhalt:

- der gesamte Umfang der Veranstaltung **ohne grobe Themeneinschränkung**
 - Lernen auf Lücke ist **nicht** möglich
 - Stoffplan Basiswissen / Kürwissen / unwichtig kurz vor der Klausur
- ca. 25% Inhaltsabfrage (z. B. Definitionen korrekt wiedergeben)
- ca. 50% Anwendung (z. B. Algorithmen von Hand nachvollziehen)
- ca. 25% Theorie (z. B. leichte Beweise)

Vortragsübersicht

1 – Einführung

- Organisation
- **Motivation**
- Inhalt der Vorlesung

- Formalisierung einfacher Daten
- maschinennahe Datenrepräsentation
- maschinenferne Datenrepräsentation

Motivation (1)

Grundbegriffe

Algorithmus (pl. Algorithmen):

Handlungsvorschrift aus endlich vielen Einzelschritten

(nach al-Chwarizmi, persischer oder iranischer Universalgelehrter, ca. 800 n. Chr)

- typische Algorithmen ermitteln aus einer Eingabe eine Ausgabe
- Algorithmen sind die Grundlage aller heutigen Computerprogramme
- Curch-Turing-These: Turing-Maschinen können alle denkbaren Algorithmen darstellen

Beispiel:

Algorithmus `telefonnummer_suchen`:

Eingabe: `name`

Ausgabe: `telefonnummer` oder `fehler`

`i = 0`

SOLANGE `existiert(eintrag(i))`:

 FALLS `name(eintrag(i)) == name`:

 ENDE, AUSGABE `telefonnummer(eintrag(i))`

`i = i + 1`

ENDE, AUSGABE `fehler`



Motivation (2)

***Datum* (pl. *Daten*, engl. *datum*, pl. *data*):**

eine gemessene oder errechnete oder erdachte Information,
die eine konzeptionelle *Semantik* (Bedeutung) besitzt

Daten können neben der Semantik diverse Eigenschaften besitzen.

Daten können zum Beispiel...

- ... *wahr* oder *falsch* sein.
- ... *real* oder *hypothetisch* sein.
- ... für einen bestimmten Anlass *relevant* (wichtig) oder *irrelevant* (unwichtig) sein.
- ... *fehlerhaft* oder *fehlerfrei* sein.
- ... *lückenhaft* oder *lückenlos* sein.
- ... *konsistent* oder *widersprüchlich* sein.

Beispiel:

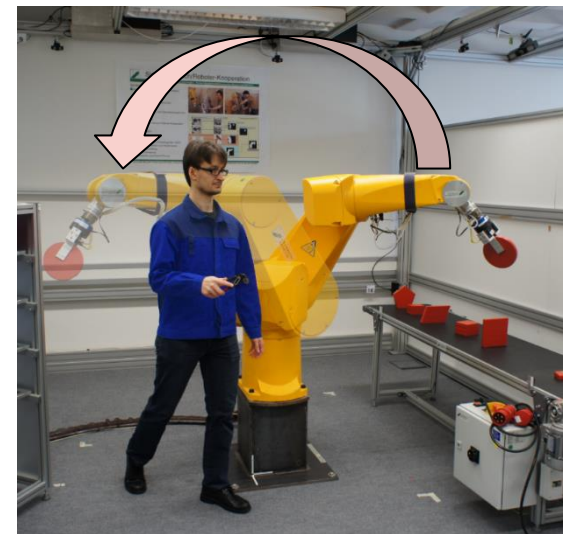
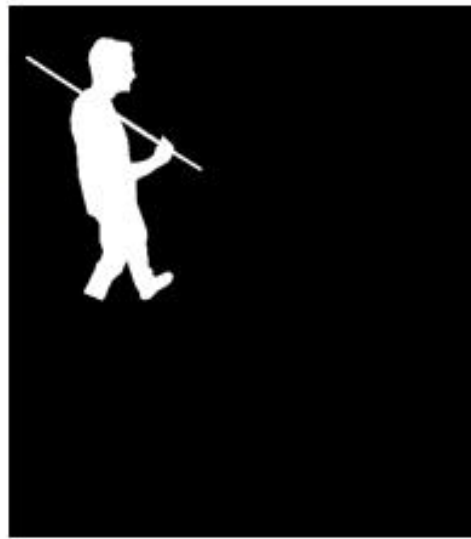
"das Kamerabild der Überwachungskamera vor Eingang 3 des Hauptgebäudes
vom 17. 03. 2025, 21:22:01" ist ein Datum mit ebenjener Semantik



Motivation (3)

Algorithmen und Daten in der Praxis

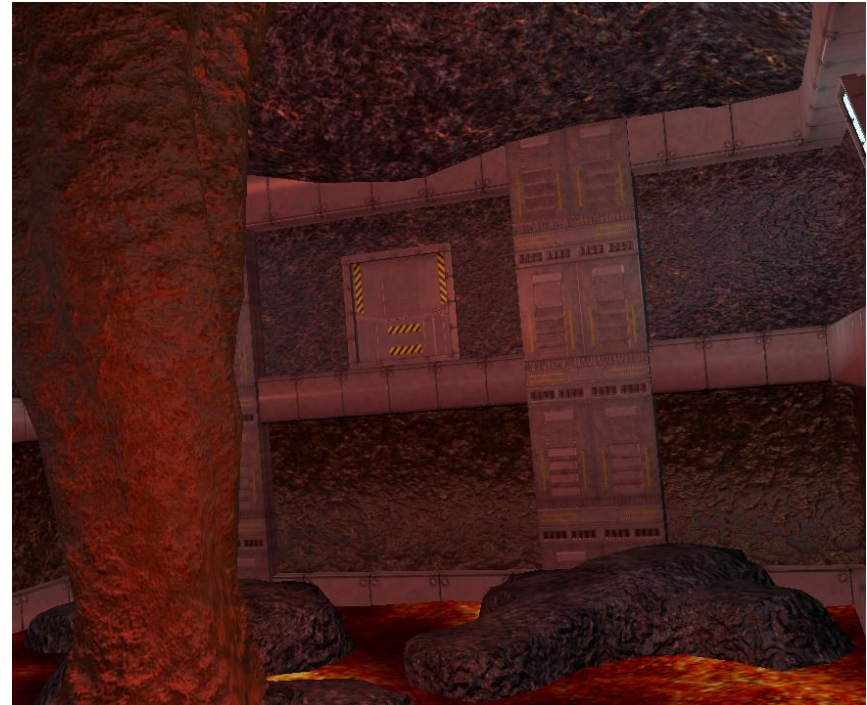
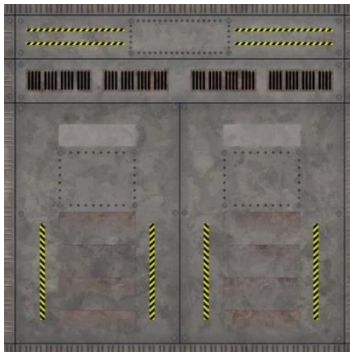
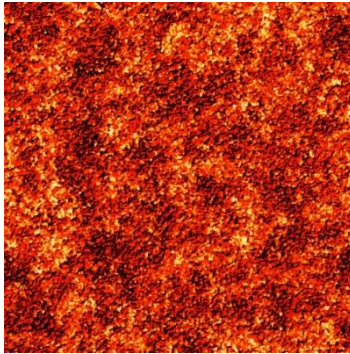
die allermeisten Anwendungsfälle in der IT (z. B. Hardware, Software) bauen darauf, dass Daten mit Algorithmen gewonnen, verarbeitet und ausgewertet werden



Beispiel Robotik:

- Sensoren gewinnen die Daten für einen Roboter
- der Roboter verarbeitet die Daten mit einem Bahnplanungsalgorithmus, um Hindernissen auszuweichen

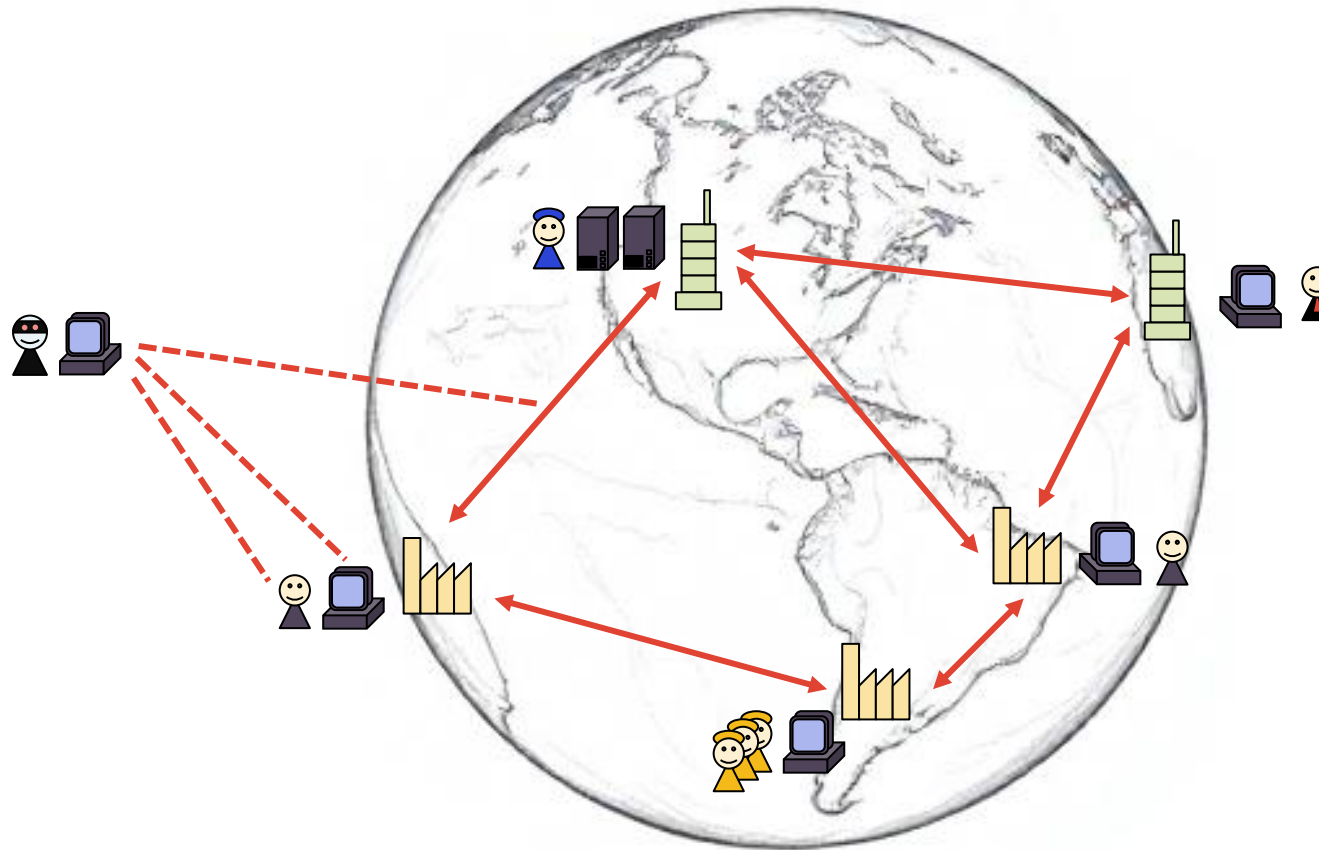
Motivation (4)



Beispiel Computerspiel:

- Künstler ersinnen Daten (z. B. Grafiken) und geben diese am Rechner ein
- die Grafikkarte verarbeitet die Daten mit einem Rasterisierungsalgorithmus, um dem Spieler eine Grafik anzuzeigen

Motivation (5)



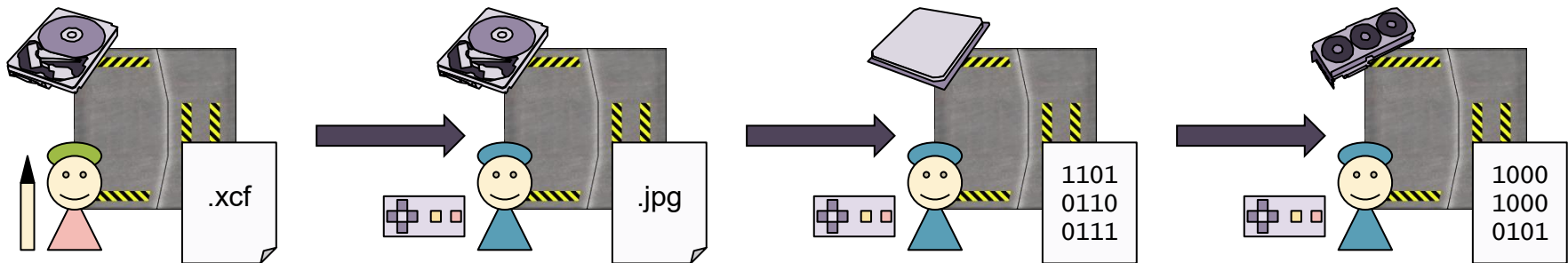
Beispiel Big Data:

- Erfassung zahlreicher Datenquellen (z. B. Handys, Produktionsstraßen, Webseitenverkehr)
- Verarbeitung im Unternehmensinteresse (z. B. Marketing, Prozessoptimierung, IT-Security)

Motivation (6)

Datenrepräsentation

- die allermeisten, digitalen Rechensysteme können ausschließlich Folgen von Bits erfassen, verarbeiten, übertragen und speichern
 - eine *Datenrepräsentation* (d. h. Instanz) eines konzeptionellen Datums ist eine Darstellung des Datums, meist als eine Folge von Bits
 - ein *Datenformat* (z. B. *Dateiformat*) gibt als begleitende Spezifikation vor,
 - ... wie viele Bits an welchem Speicherort zu dem Datum gehören, ...
 - ... welche Regeln (z. B. *Syntax*) die Bits einhalten müssen, und...
 - ... welchen Bezug (z. B. *Semantik*) die einzelnen Bits zu dem Datum besitzen
- ein einzelnes, konzeptionelles Datum kann mehrere Datenrepräsentationen besitzen
- die Datenrepräsentation kann sich im Verlauf der Zeit mit oder ohne Verlust ändern



Motivation (7)

Datenaufbereitung

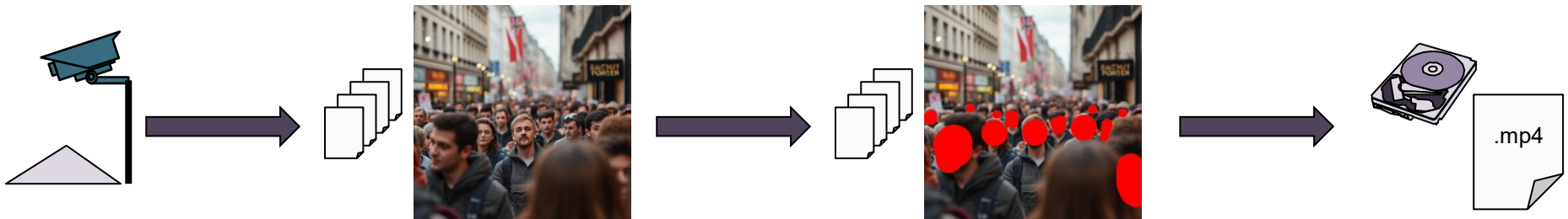
die *Datenaufbereitung* bestimmt aus Daten (bzw. deren Repräsentationen) weitere Daten (bzw. deren Repräsentationen), wobei die errechneten Daten für nachfolgende Verarbeitungsschritte besser geeignet sind

→ *Rohdaten*: weitestgehend ursprüngliche Datenrepräsentation (z. B. wie direkt vom Sensor)

→ *abgeleitete Daten*: aus anderen Daten (z. B. aus Rohdaten) errechnete Daten

zur Datenaufbereitung gehört unter anderem:

- Erkennen und Auffüllen lückenhafter Daten
- Erkennen und Verwerfen unwichtiger Daten
- Erkennen und Bereinigen von Fehlern und Widersprüchen in den Daten
- Ändern des Datenformats oder der Datenrepräsentation
- Sammeln oder Kombinieren mehrerer Daten zu einem gemeinsamen Datum



Motivation (8)

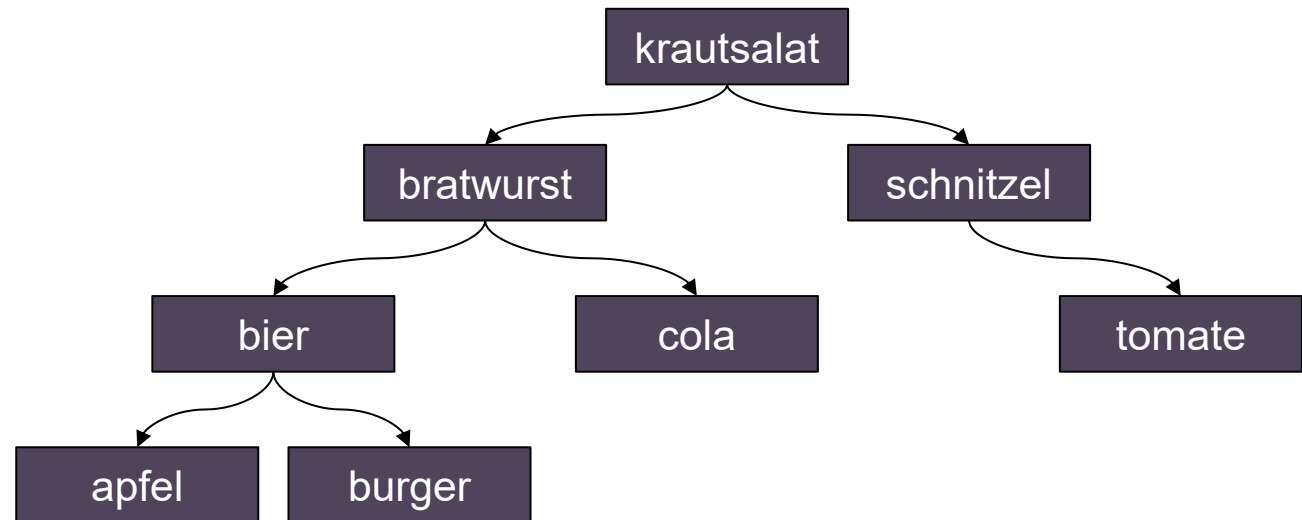
Datenstruktur

eine *Datenstruktur* ist eine Vorgabe, wie Daten und deren Zusammenhänge in Datenrepräsentationen umgesetzt (z. B. im Speicher angeordnet) werden

Beispiele:

1	apfel
2	schnittzel
3	krautsalat
4	bratwurst
5	burger
6	tomate
7	cola
8	bier

Array



binärer Suchbaum

Motivation (9)

Operationen (auf Datenstrukturen)

Algorithmen, welche die spezielle Anordnung von Daten in einer Datenstruktur nutzen, um ein bestimmtes Ergebnis zu erzielen (z. B. prüfen, ob ein Wert abgespeichert ist)

typische Operationen:

- Anlegen und Löschen der Datenstruktur
- Einfügen neuer Daten in die Datenstruktur
 - am Anfang, am Ende, an vorgegebener Stelle, an richtiger Stelle
- Löschen von Daten aus der Datenstruktur
- von einem Datum der Datenstruktur zum nächsten oder vorigen Datum wechseln
- das Datum an n -ter Stelle in der Datenstruktur auslesen
- feststellen, ob ein gegebenes Datum in der Datenstruktur enthalten ist...
- ... und dieses Datum auslesen

Motivation (10)

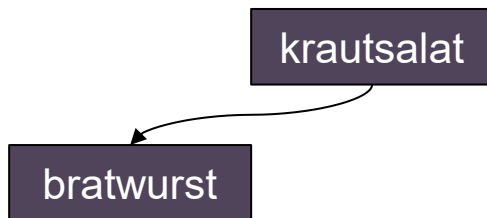
Verweise

zwischen Daten können *Verweise* (d. h. Bezüge) bestehen, die in der Regel selbst ein (Teil-)Datum mit Semantik sind
→ Darstellung meist als Pfeil mit Quelle und Ziel

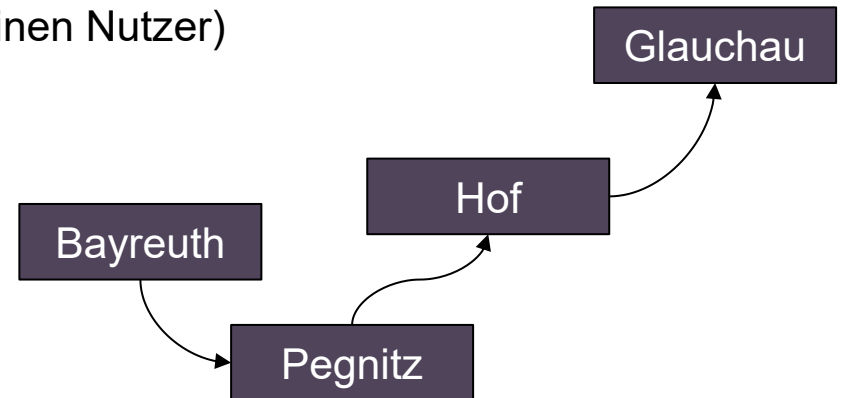
tendenziell beschäftigt man sich nur mit relevanten Verweisen:

- Verweise mit interner Relevanz (z. B. für einen Algorithmus)
- Verweise mit Anwendungsrelevanz (z. B. für einen Nutzer)

Beispiele:



intern relevanter Verweis:
speichere Sortierung von Worten ab



anwendungsrelevanter Verweis:
speichere mögliche Bahnverbindungen ab

Vortragsübersicht

1 – Einführung

- Organisation
- Motivation
- Inhalt der Vorlesung

- Formalisierung einfacher Daten
- maschinennahe Datenrepräsentation
- maschinenferne Datenrepräsentation

Inhalt der Vorlesung (1)

Prämisse

Algorithmen und Datenstrukturen sind die wichtigste Kernkompetenz von Informatikern

- tägliche Anwendung bei Entwurf, Umsetzung, Wartung von Software
- man muss zahlreiche, erprobte Algorithmen und Datenstrukturen kennen
- typisches Thema bei Bewerbungsgesprächen

typische Problemstellungen im Berufsleben:

- Wähle einen erprobten Algorithmus, eine erprobte Datenstruktur für ein vorgegebenes Problem aus.
- Finde heraus, ob es eine passende Implementierung bereits gibt und wende diese gegebenenfalls an.
- Erstelle eine neue Implementierung für erprobte Algorithmen oder Datenstrukturen für ein konkretes Anwendungsproblem.
- Erfinde einen neuen Algorithmus oder eine neue Datenstruktur, **wenn und nur wenn** es noch nichts speziell passendes gibt (und nutze erprobte Algorithmen, Datenstrukturen als Inspiration).



Inhalt der Vorlesung (2)

Lernziele

erprobte Algorithmen und Datenstrukturen:

- Grundkonzepte
- Zusammenhänge:
 - Welcher Algorithmus auf welcher Datenstruktur?
- Bewertung:
 - Wann welcher Algorithmus?
 - Wann welche Datenstruktur?
- rudimentäre Implementierung in C++
 - Evaluation der eigenen Lösung
 - Einordnung in die theoretischen Erkenntnisse

Abgrenzung:

- keine Beweise, keine Mathematik
- nur kleine, selbstgeschriebene Programme
- keine umfangreichen Anwendungen außerhalb der Theorie

Inhalt der Vorlesung (3)

Geplante Themen und Vorlesungsstruktur

1 – objektorientierte Programmierung in C++

- Wie lassen sich Objekte der anschaulichen Vorstellung am Rechner abbilden?
- Worauf muss man speziell bei C++ achten?

2 – grundlegende Algorithmen und Datenstrukturen

- Wie kann man Daten ohne größere Struktur handhaben (z. B. sortieren)?
- In welcher Datenstruktur lassen sich beliebige Daten unterbringen?

3 – Baumdatenstrukturen

- Wie kann man Daten in einer Hierarchie (z. B. Eltern-Kind-Beziehung) verwalten?
- Wie lässt sich verhindern, dass die Hierarchie zu einer Liste degeneriert

4 – Hashverfahren

- Wie kann man Daten über mehr oder minder eindeutige Identifikatoren verwalten?

5 – Graphen

- Wie kann man beliebige Zuordnungen zwischen Daten abbilden und auswerten?

Vortragsübersicht

1 – Einführung

- Organisation
- Motivation
- Inhalt der Vorlesung

- Formalisierung einfacher Daten
- maschinennahe Datenrepräsentation
- maschinenferne Datenrepräsentation

Formalisierung einfacher Daten (1)

Daten und Datentypen

- verwende für $|D|$ konzeptionelle Daten eindeutige Namen d_i mit $i = 1, \dots, |D|$
- verwalte alle Namen in einer Menge $D = \{d_1, d_2, \dots, d_{|D|}\}$
- gebe für jedes konzeptionelle Datum d_i als Datentyp eine Menge zulässiger Werte W_i vor
 - am Rechner ist jedes W_i zwingend endlich, in der realen Welt (vermutlich) nicht
 - bei Bedarf lassen sich in W_i Einträge für Spezialfälle (z. B. `nullptr`) vorsehen
- modelliere den Wert $w_i \in W_i$ zu jedem Datum über eine Funktion $f: D \rightarrow W, f(d_i) = w_i$
 - diese Zuordnung ist aufgrund der Mathematik statisch
 - veränderliche Größen ergeben bei jeder Erfassung ein neues Datum
(z. B. könnte i die Zeit bei der Messung wiedergeben)

Beispiel: Sei $D = \{d_1, d_2, d_3\}$.

Hier ist d_1 eine Matrikelnummer, d_2 die Prozentzahl in einer Prüfung, d_3 die erzielte Note.

Dann könnte man $W_1 = \mathbb{N}$, $W_2 = [0, 100]$ und $W_3 = \{1, 1.3, 1.7, 2.0, \dots, 4.0, \text{n.b.}\}$ wählen,

ferner $f(d_1) = 10023$, $f(d_2) = 93$ und $f(d_3) = 1.3$ festlegen.

Formalisierung einfacher Daten (2)

Zusammengesetzte Daten

- setze $k = |d_j|$ Einzeldaten $d_{i_1}, d_{i_2}, \dots, d_{i_k}$ zu einem Datum $d_j = (d_{i_1}, d_{i_2}, \dots, d_{i_k})$ zusammen
 $\rightarrow d_{i_1} \in D, d_{i_2} \in D, \dots, d_{i_k} \in D, d_j \in D$
- Wertebereich des Gesamtdatums $W_j = W_{i_1} \times W_{i_2} \times \dots \times W_{i_k}$
- **Achtung:** Wertezuordnungsfunktion f muss konsistent bleiben,
d. h. $f(d_j) = f(d_{i_1}, d_{i_2}, \dots, d_{i_k}) = (f(d_{i_1}), f(d_{i_2}), \dots, f(d_{i_k}))$

Beispiel:

Sei $D = \{d_1, d_2, d_3, d_4\}$, ferner d_1, d_2, d_3 und W_1, W_2, W_3 wie zuvor, schließlich weiterhin $f(d_1) = 10023$, $f(d_2) = 93$ und $f(d_3) = 1.3$.

Dann könnte man $W_4 = W_1 \times W_2 \times W_3$ setzen und $d_4 = (d_1, d_2, d_3)$ als zusammengesetztes Datum ("ein Datensatz") vorgeben.

Damit wäre $f(d_4)$ nur möglich als

$$f(d_4) = f(d_1, d_2, d_3) = (f(d_1), f(d_2), f(d_3)) = (10023, 93, 1.3).$$

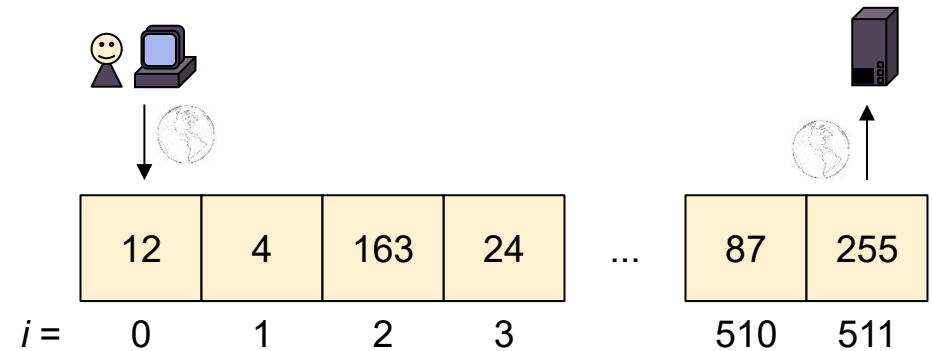
Formalisierung einfacher Daten (3)

Unstrukturierte Daten

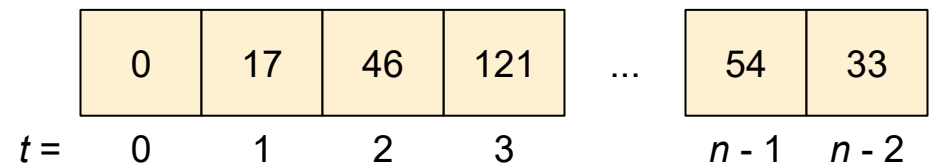
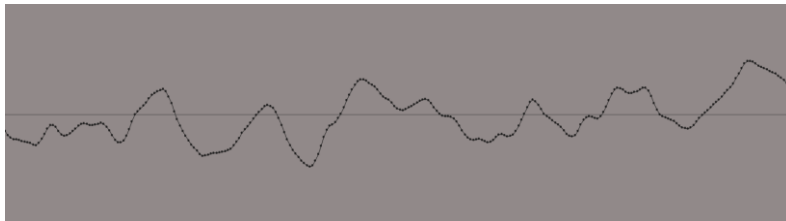
- die Daten liegen in einer (weithin) unspezifizierten, oft flachen Datenstruktur vor (z. B. Folge von Bytes, Arrays mit einem einzelnen Datentypen)
- zwischen einzelnen Daten gibt es in der Regel wenige oder keine Verweise, welche durch die Struktur oder als zusätzliche Daten vorgegeben werden

byteweises Protokollieren verschlüsselter Daten

```
std::vector<unsigned char> proxy_buffer;  
connection_in.read(network_buffer, 512);  
connection_out.write(network_buffer, 512);
```



Ablegen eines Audiosignals im Arbeitsspeicher

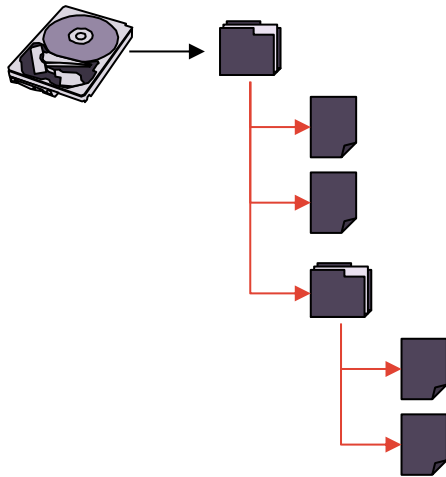


Formalisierung einfacher Daten (4)

Strukturierte Daten

- die Daten liegen in einer (mehr oder minder) spezifizierten, höheren Datenstruktur vor (z. B. Liste, Baum, Tabelle mit spezifizierten Spalten, allgemeiner Graph)
- zwischen den Daten existieren häufig durch die Datenstruktur implizit vorgegebene und/oder außerhalb der Struktur ausdrücklich modellierte **Verweise**

als Dateien
im Dateisystem



als Datensätze in einer Datenbank (z. B. SQL)

TABLE bestellung

name	lieferort	bstnr
werner	134	86989
werner	134	74512
lobachev	3	86989
ehrlich	209	90627

TABLE katalog

artnr	artikel	lagerort
86989	taster	1
74512	maus	4
90627	monitor	2

TABLE ort

ortnr	adresse
1	finsterweg 5
2	goldene allee
3	pflastergasse

Vortragsübersicht

1 – Einführung

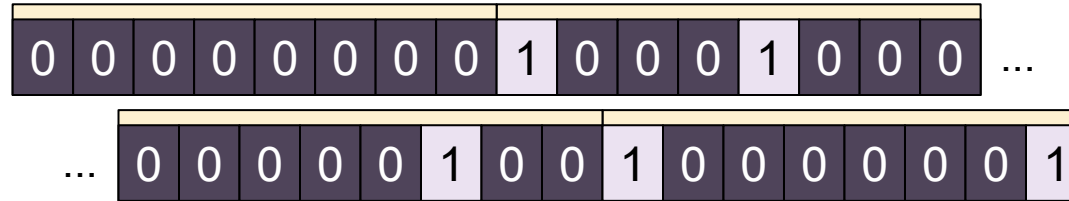
- Organisation
- Motivation
- Inhalt der Vorlesung

- Formalisierung einfacher Daten
- **Maschinennahe Datenrepräsentation**
- Maschinenferne Datenrepräsentation

Maschinennahe Datenrepräsentation (1)

Maschinennahe Repräsentation einzelner Daten

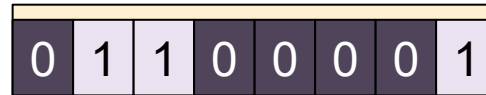
int i = 8914049;



int hat (hier)
32 Bit

$$8914049 = 1 * 2^{23} + 1 * 2^{19} + 1 * 2^{10} + 1 * 2^7 + 1 * 2^0$$

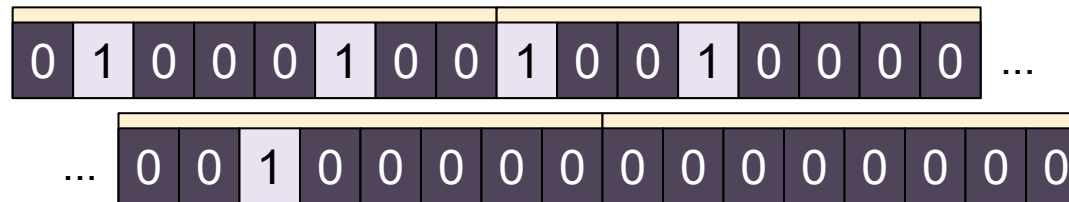
char c = 'a';



char hat (hier)
8 Bit

$$'a' = 97 = 1 * 64 + 1 * 32 + 1 * 1$$

float f = 1153.f;

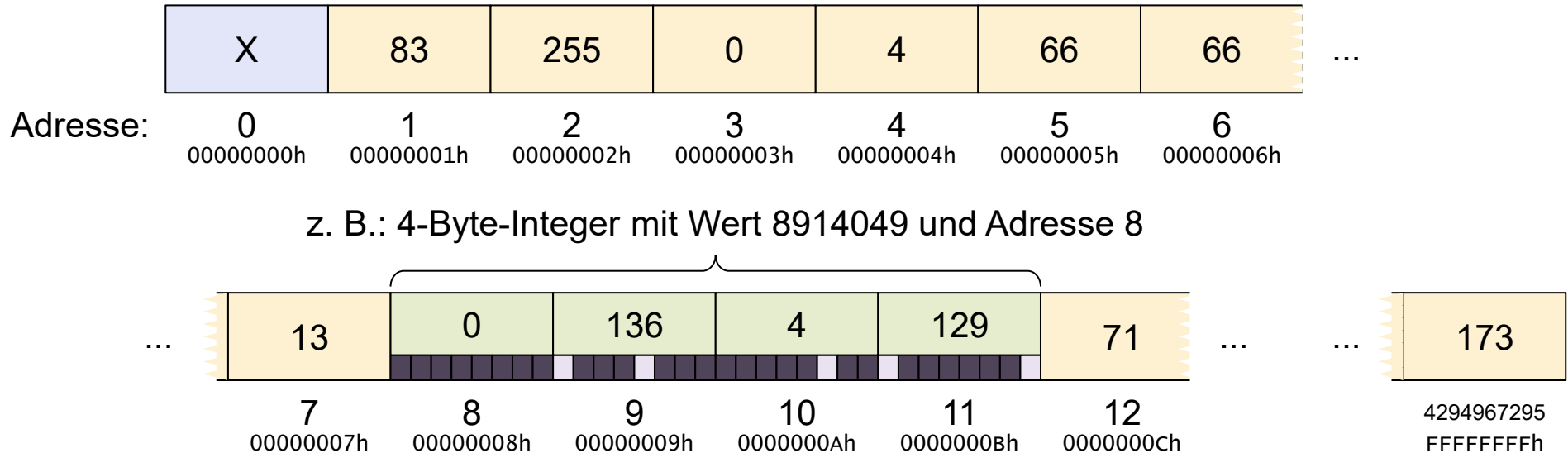


IEEE-float hat
32 Bit,
Umrechnung
ist komplizierter!

- Rechner merken sich Einzeldaten in einer *binären Repräsentation*:
 - *Datentyp* als Datenformat bestimmt Anzahl an Bits und deren Semantik
 - jedes Bit kann entweder den Wert 0 oder den Wert 1 annehmen
 - alle Bits zusammen speichern die Binärrepräsentation des Einzeldatums

Maschinennahe Datenrepräsentation (2)

Maschinennahe Repräsentation in Speicherzellen



- Ausblick *abstraktes Speichermodell* in VL Betriebssysteme:
 - Speicher (z. B. Hauptspeicher, Datenträger, Datei) ist lineare Folge von Speicherzellen
 - *Adresse* einer Speicherzelle ist deren *Index* (Abstand vom Start) im Speichermodell
 - jede Speicherzelle fasst 1 Byte (d. h. 8 **nicht einzeln adressierbare** Bits)
- jede hardwarenahe Repräsentation eines Datums belegt eindeutige Speicherzellen
 - nutze (per Konvention) die niedrigste Adresse einer Repräsentation
als Adresse der gesamten Repräsentation (vgl. C++: &i von int-Variable i)

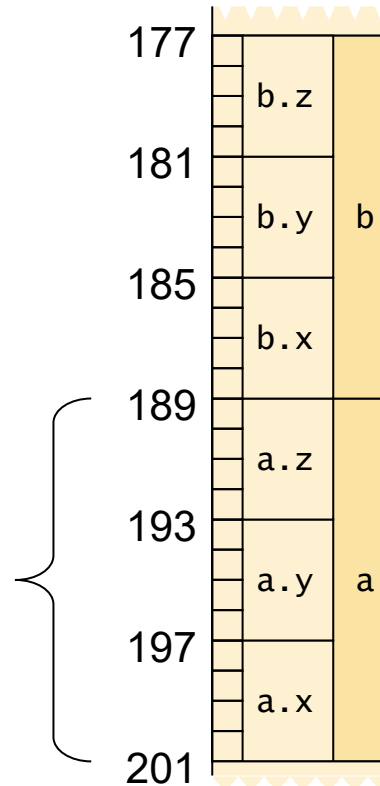
Maschinennahe Datenrepräsentation (3)

Maschinennahe Repräsentation zusammengesetzter Daten

```
struct point3f
{
    float x;
    float y;
    float z;
};

void f()
{
    point3f a;
    point3f b;
}
```

zusammengesetztes Datum
als binäre Repräsentation



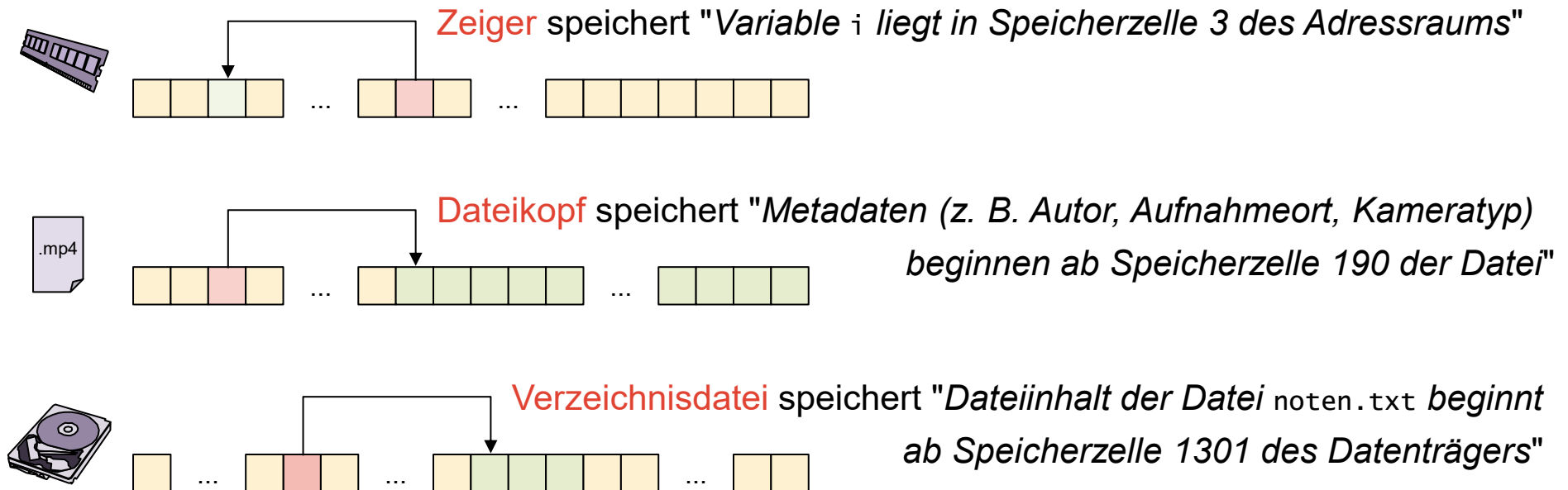
Stapel des C-Programms
(Teil des Hauptspeichers,
hier vertikal dargestellt)

einzelnes Teildatum
als binäre Repräsentation

- *zusammengesetzte Daten* fassen mehrere, gesonderte Daten zusammen
→ naive, hardwarenahe Repräsentation am Rechner durch eine sequentielle Reihung der Repräsentationen der Einzeldaten in aufeinanderfolgenden Speicherzellen

Maschinennahe Datenrepräsentation (4)

Maschinennahe Repräsentation von Verweisen



- Konzept *Zeiger* (C, C++) bzw. *Referenz* (C++, Java, C#):
 - speichere Adresse (z. B. im Hauptspeicher) für einen Verweis in anderem Datum ab
- analoges Vorgehen bei anderen, linear durchzählbaren Speicherarten
 - Adresse relativ zu erster Zelle von Datenträger, Datei, Netzwerktransfer
 - Index in einem (ggf. linearisierten) Array

Maschinennahe Datenrepräsentation (5)

Bewertung maschinennaher Datenrepräsentation

Vorteile:

- schnelles Rechnen, schneller Verweiszugriff
 - geringerer Platzbedarf
- } jeweils ca. Faktor 5 ... 1000
gegenüber anderen Repräsentationen

Nachteile:

- Darstellung für normale Menschen ohne Aufbereitung schwer nachzuvollziehen
- Repräsentation ist oft maschinen- und mitunter prozessabhängig (z. B. Speicherlayout)
→ Konvertierung zwischen Prozessen und Maschinen nötig, sonst Datenkorruption

Beispiel:

- *Byte Endianness* (auch *Byte Order*, dt. *Byte-Reihenfolge*) gibt an, ob das erste Byte (*big endian*) oder das letzte Byte (*little endian*) den höchsten Beitrag zu dem Wert eines Integers liefert

DE	AD	00	00
----	----	----	----

= 3735879680

big endian-int

DE	AD	00	00
----	----	----	----

= 57005

little endian-int

Vortragsübersicht

1 – Einführung

- Organisation
- Motivation
- Inhalt der Vorlesung

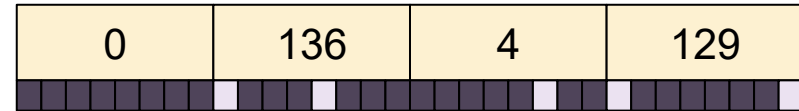
- Formalisierung einfacher Daten
- Maschinennahe Datenrepräsentation
- Maschinenferne Datenrepräsentation

Maschinenferne Datenrepräsentation (1)

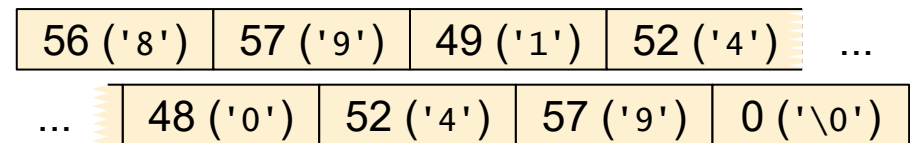
Maschinenferne Repräsentation einzelner Daten

Ganzzahl-Datum mit Wert 8914049...

... als maschinennahe Repräsentation
mit 4-Byte-`int`

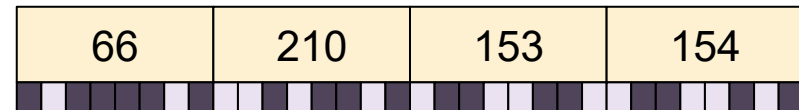


... als maschinenferne Repräsentation
mit ASCII-Folge aus 8 Zeichen zu 1 Byte



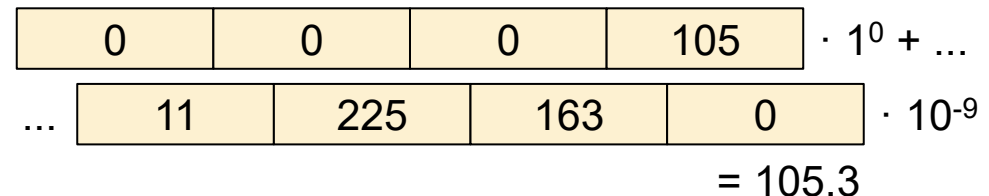
reelles Datum mit Wert 105,3...

... als maschinennahe Repräsentation
mit 4-Byte-`float` (→ **Rundungsfehler!**)



≈ 105.300003

... als maschinenferne Repräsentation
mit Festkomma über 2 binäre `ints`



- spezifiziere nach Bedarf eine maschinenunabhängige Repräsentation
 - typisch: Datenrepräsentation als menschenlesbarer ASCII-Text
 - seltener: Datenrepräsentation als maschinenunabhängiges Binärformat

Maschinenferne Datenrepräsentation (2)

Maschinenferne Repräsentation zusammengesetzter Daten

Beispiel .csv-Format (*comma separated values*)

```
Matrikelnummer,Prozent,Note
15001194,57,3.7
15002378,51,4
15004369,40,n.b.
15010853,75,2.3
15015471,35,n.b.
15015518,100,1
15017645,7,n.b.
15019962,64,3.3
15020413,93,1.3
15020795,88,1.7
15023820,71,2.7
```

Beispiel .xml-Format (*Extensible Markup Language*)

```
<?xml version="1.0" encoding="UTF 8">
<notenliste>
  <eintrag Matrikelnummer="15001194"
    Prozent="57"
    Note="3.7" />
  <eintrag Matrikelnummer="15002378"
    Prozent="51"
    Note="4" />
  <eintrag Matrikelnummer="15004369"
    Prozent="40"
    Note="n.b." />
  <!-- ... -->
</notenliste>
```

Beispiel .json-Format (*JavaScript object notation*)

```
[
  {
    "Matrikelnummer": 15001194,
    "Prozent": 57,
    "Note": 3.7
  },
  {
    "Matrikelnummer": 15002378,
    "Prozent": 51,
    "Note": 4
  },
  {
    "Matrikelnummer": 15004369,
    "Prozent": 40,
    "Note": "n.b."
  },
  ; ...
]
```

- spezifiziere nach Bedarf ein größeres Datenformat
 - oft (weitgehend) menschenlesbarer ASCII-Text
 - meist zusätzliche, strukturierende Bestandteile (z. B. Schlüsselworte, Interpunktion, Weißraum)

Maschinenferne Datenrepräsentation (3)

Maschinenferne Repräsentation von Verweisen

Grundidee:

generiere für eine Datenrepräsentation eine im Verwendungskontext eindeutige *ID*

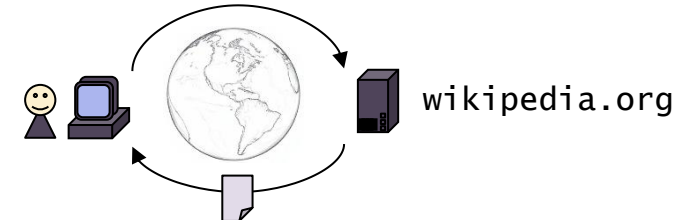
→ gebe in Verweisen die ID statt der Repräsentation an

→ *lookup*: für Zugriffe muss das System zur ID die Repräsentation lokalisieren

Varianten:

- Vorgabe (z. B. zum Teil) durch den Menschen
- maschinelle Erzeugung durch Algorithmus
- randomisierte Erstellung

wiki/Universally_Unique_Identifier



Beispiele:

- *UUID* (**u**niversally **u**nique **i**dentifier) bzw. *GUID* (**g**lobally **u**nique **i**dentifier)
verwendet 128-Bit-Zahl als ID, je nach Vergabeverfahren zum Teil zufallsgeneriert
- *URI* (**u**niform **r**esource **i**dentifier) verwendet eindeutige Zeichenfolge als ID
 - *ISBN* (**i**nternational **s**tandard **b**ook **n**umber) zur Identifikation von (Print-)Büchern
 - *URL* (**u**niform **r**esource **l**ocator) gibt Zugriffsmethode und (je nach Methode) Ort an

Maschinenferne Datenrepräsentation (4)

Bewertung maschinenferner Datenrepräsentation

Vorteile:

- Austausch zwischen Prozessen, Speichertypen, Rechnern unproblematisch
- von Menschen (mehr oder minder) gut zu prüfen

Nachteile:

- Zusatzbedarf an Rechenzeit
 - komplexe Algorithmen oder Umrechnung in maschinennahe Formate
 - Lookup für Verweise nötig
- Zusatzbedarf an Speicherplatz
 - implizite Redundanz ermöglicht Inkonsistenzen (d. h. mehr Fehlerquellen)

Beispiel:

	Ganzzahl in string a				Ganzzahl in string b		
Ganzzahl-Addition	49 ('1')	53 ('5')	0 ('\0')	+	55 ('1')	51 ('3')	0 ('\0')

→ naiver Ansatz: 13 Rechenoperationen bzw. Zugriffe

$$(a[0] - 48) * 256 + (a[1] - 48) + (b[0] - 48) * 256 + (b[1] - 48)$$

→ vgl. für `int a` und `int b` ist `a + b` eine Rechenoperation

Zusammenfassung

Grundbegriffe rund um Daten

- Algorithmus, Datum, Datenstruktur
- Datenrepräsentation, Datenformat (bzw. Dateiformat)
- Rohdaten, abgeleitete Daten
- Datenaufbereitung als vorbereitender Verarbeitungsschritt

Handhabung einfacher Daten

- Formalisierung von Daten als Elemente aus einer Menge
- maschinennahe Repräsentation von Daten
 - speichereffizient, laufzeiteffizient, wenig redundant
 - eindeutige Verweise einfach zu generieren und zu verfolgen
 - schwer menschenlesbar, schwer über Systeme tauschbar
- maschinenferne Repräsentation von Daten
 - höhere Speicher- und Laufzeitkosten, höhere Redundanz
 - eindeutige Verweise schwerer zu generieren und zu verfolgen
 - oft menschenlesbar, leicht über Systeme tauschbar